I. BEYOND SINGLE TABLES: DATA INTEGRATION

In real-world Big Data environments, information is rarely stored in a single file. For instance, student demographics might be in one database, while exam scores are in another. To perform a comprehensive analysis, a Data Engineer must merge these sources.

1.1. Concatenation (pd.concat)

Concatenation is used to "glue" DataFrames together along an axis (either stacking them vertically or placing them side-by-side).

- **Vertical Stack:** Combining monthly sales logs into a yearly report.
- **Horizontal Stack:** Adding new feature columns to an existing dataset.

1.2. Merging and Joining (pd.merge)

Merging is the Pandas equivalent of **SQL Joins**. It aligns rows based on one or more keys (IDs).

- **Inner Join:** Returns only the records with matching keys in both tables.
- **Left Join:** Keeps all records from the "Left" table and adds matching data from the "Right" table (Essential for maintaining the integrity of a primary dataset).
- **Outer Join:** Returns all records from both tables, filling missing values with NaN.

II. THE POWER OF GROUPBY: SPLIT-APPLY-COMBINE

The groupby() function is perhaps the most powerful tool in the Pandas library. it follows a three-step process to summarize data:

1. **Split:** Breaking the data into groups based on some criteria (e.g., grouping students by "Major").
2. **Apply:** Computing a function within each group (e.g., calculating the "Average GPA").
3. **Combine:** Stitching the results back into a new data structure.

2.1. Common Aggregation Functions

- .mean(), .sum(), .count()
- .min(), .max()
- .std() (Standard Deviation for variance analysis)

2.2. Multi-Level Aggregation

Using .agg(), you can apply different functions to different columns simultaneously. For example, you can find the *average* score but the *maximum* attendance for each class in one line of code.

III. PIVOT TABLES AND CROSS-TABULATIONS

For business intelligence and reporting, DataFrames can be reshaped into **Pivot Tables**, similar to those found in Microsoft Excel but capable of handling much larger volumes of data.

- **df.pivot_table():** Allows for multi-dimensional grouping. For example, showing "Average Salary" categorized by both "Department" (Rows) and "City" (Columns).

- **Cross-Tabs (pd.crosstab):** Specifically used for frequency tables, such as counting how many students from each high school passed the DTU entrance exam.

IV. TIME-SERIES WRANGLING

Big Data often involves timestamps (log files, stock prices, sensor data). Pandas was originally developed for financial time-series, making it elite at handling dates.

- **Datetime Conversion:** pd.to_datetime() converts strings into actual timestamp objects.
- **Resampling:** Changing the frequency of data. For example, taking "Minutely" sensor data and converting it into "Daily" averages.
- **Rolling Windows:** Calculating moving averages to smooth out noise in data trends.

V. PRACTICAL LAB: THE ANALYTICS PIPELINE

Python

```
import pandas as pd


# 1. Integration: Merging Student Info with Exam Results

students = pd.read_csv("students.csv")

exams = pd.read_csv("results.csv")

df = pd.merge(students, exams, on="student_id", how="left")


# 2. Aggregation: Insights by Major

report = df.groupby("major").agg({

    "gpa": "mean",

    "student_id": "count"

}).rename(columns={"student_id": "total_students"})


# 3. Pivot: Grades by Major and Semester

pivot = df.pivot_table(values="score", index="major", columns="semester", agg_mean="mean")
```